

GLOBAL: DESARROLLO DE SOFTWARE

Nicolás Moreno – 50989 – 3k9

Link del

repositorio: <https://github.com/NicolasM45414/GlobalMutantesMercadoLibre3k9.git>

Link del deploy: <https://globalmutantesmercadolibre3k9.onrender.com>

Introducción

El propósito de este desarrollo es construir una API REST de alto rendimiento diseñada para identificar si un individuo posee características mutantes a partir del análisis de su cadena de ADN. La aplicación procesa una estructura matricial de secuencias genéticas (NxN), examina patrones característicos y persiste los hallazgos para la generación de métricas de validación.

La implementación se ha realizado empleando Java 17 junto con Spring Boot 3, con despliegue en infraestructura cloud a través de Render y empaquetado mediante contenedores Docker.

Arquitectura y Diseño

El sistema adopta una **Arquitectura por Capas (Layered Architecture)** con el fin de garantizar la separación clara de responsabilidades, facilitar el mantenimiento y potenciar la escalabilidad del proyecto.

3.1. Controlador (MutanteController)

Constituye la interfaz de entrada de la API. Su responsabilidad consiste en recibir las solicitudes HTTP, aplicar validaciones sobre el formato de los datos mediante anotaciones (`@Valid`, `@NotNull`, `@ValidDnaSequence`) y transferir la lógica de procesamiento hacia la capa de servicios.

- **Documentación integrada:** Incorpora especificación OpenAPI/Swagger mediante anotaciones (`@Operation`, `@ApiResponse`) para simplificar la integración con la API.

- **Gestión de respuestas HTTP:** Administra los códigos de estado conforme a los requerimientos establecidos: **200 OK** cuando se identifica un mutante y **403 Forbidden** para individuos humanos.

Servicio (MutanteService)

Alberga la lógica central del dominio. Aplica una estrategia de optimización fundamentada en **Hashing y Almacenamiento en Caché** para prevenir el reprocesamiento de secuencias de ADN previamente analizadas:

1. **Cálculo de Hash:** Se genera un identificador único mediante algoritmo SHA-256 sobre la secuencia de ADN recibida.
2. **Consulta en Repositorio:** Previo a la ejecución del algoritmo de detección de patrones, se verifica en la base de datos la existencia de dicho hash.
3. **Procesamiento:** En caso de tratarse de una secuencia nueva, se invoca al MutanteDetector para realizar la búsqueda de patrones.
4. **Almacenamiento:** El resultado obtenido se registra en la base de datos H2 vinculado al hash generado, garantizando la unicidad de cada registro de ADN.

DTOs y Validación

Se emplean objetos de transferencia de datos (Data Transfer Objects - **DnaRequest**) con el objetivo de desacoplar la capa de presentación del modelo de persistencia.

Se han implementado validaciones customizadas (**@ValidDnaSequence**) que aseguran la cuadratura de la matriz y la presencia exclusiva de caracteres válidos (A, T, C, G) previo a cualquier operación de procesamiento.

Algoritmo de Detección

La identificación de mutantes se ejecuta mediante el recorrido sistemático de la matriz de ADN (representada como un array de Strings) en búsqueda de múltiples secuencias compuestas por cuatro caracteres idénticos consecutivos.

Estrategia de Exploración:

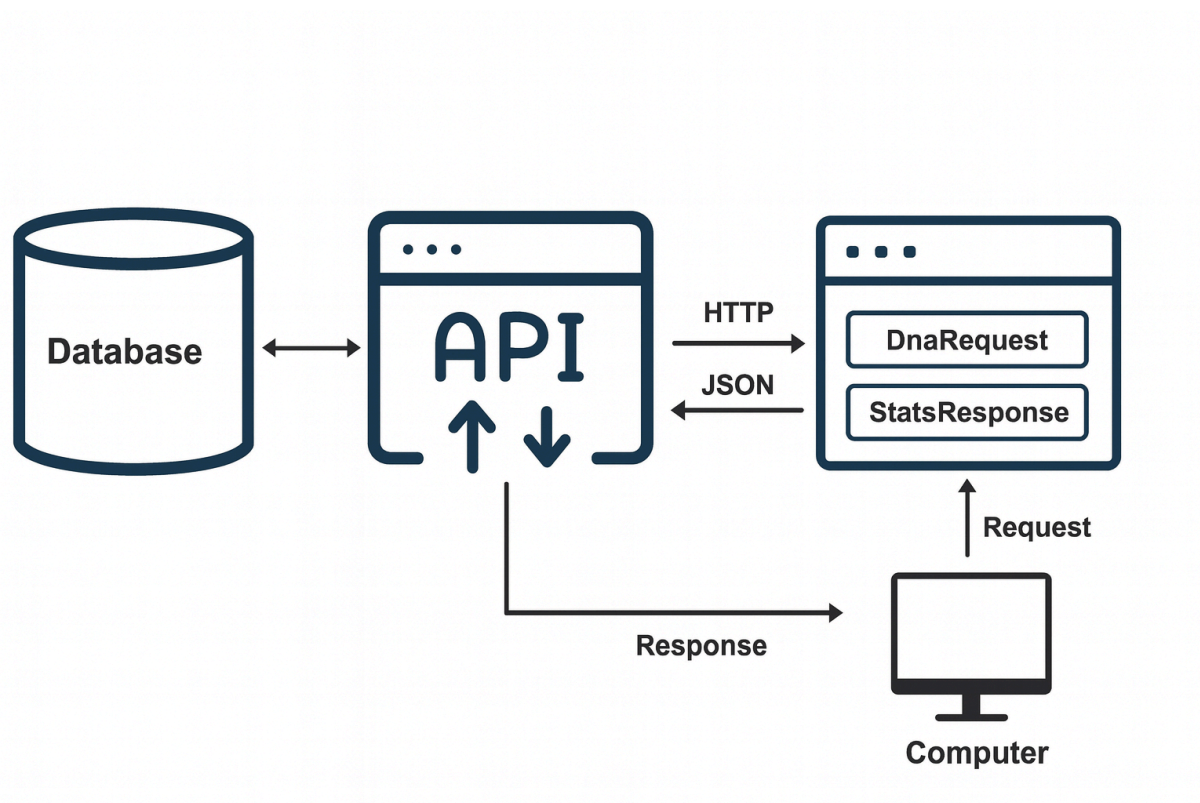
El algoritmo inspecciona la matriz siguiendo tres vectores direccionales:

1. **Horizontal:** Exploración por filas en sentido izquierda a derecha.
2. **Vertical:** Recorrido por columnas de arriba hacia abajo.
3. **Diagonal:** Análisis de diagonales tanto principales como secundarias.

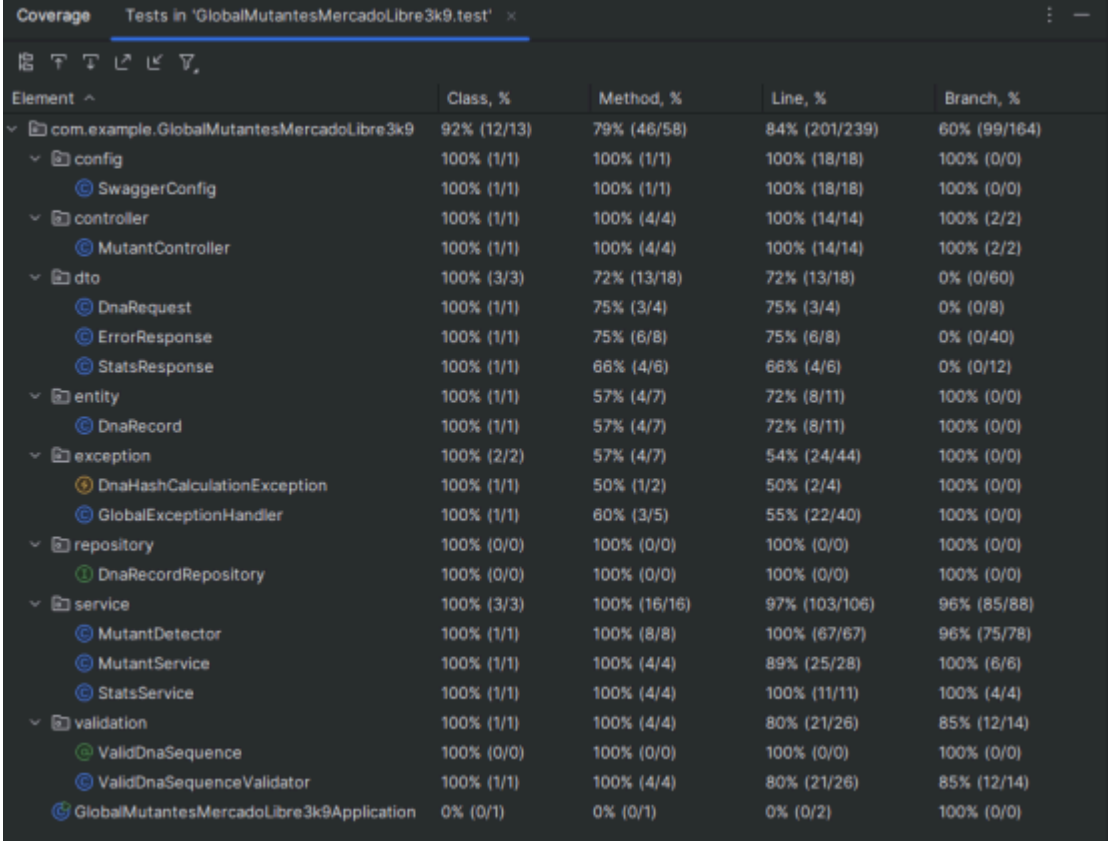
Optimizaciones:

- **Salida Temprana (Early Exit):** El algoritmo mantiene un contador de secuencias detectadas. Una vez que el contador supera el valor de 1, la ejecución se interrumpe inmediatamente retornando **true**, eliminando iteraciones superfluas sobre el resto de la matriz.
- **Validación Estructural:** Se verifica la integridad dimensional de la matriz (NxN) antes de su procesamiento para prevenir excepciones de índice fuera de rango (**IndexOutOfBoundsException**).

Arquitectura del sistema



Code Coverage:



The screenshot shows the Coverage tool in IntelliJ IDEA, displaying code coverage for the test 'GlobalMutantesMercadoLibre3k9.test'. The table lists various code elements and their coverage percentages.

Element	Class, %	Method, %	Line, %	Branch, %
com.example.GlobalMutantesMercadoLibre3k9	92% (12/13)	79% (46/58)	84% (201/239)	60% (99/164)
config	100% (1/1)	100% (1/1)	100% (18/18)	100% (0/0)
SwaggerConfig	100% (1/1)	100% (1/1)	100% (18/18)	100% (0/0)
controller	100% (1/1)	100% (4/4)	100% (14/14)	100% (2/2)
MutantController	100% (1/1)	100% (4/4)	100% (14/14)	100% (2/2)
dto	100% (3/3)	72% (13/18)	72% (13/18)	0% (0/60)
DnaRequest	100% (1/1)	75% (3/4)	75% (3/4)	0% (0/8)
ErrorResponse	100% (1/1)	75% (6/8)	75% (6/8)	0% (0/40)
StatsResponse	100% (1/1)	66% (4/6)	66% (4/6)	0% (0/12)
entity	100% (1/1)	57% (4/7)	72% (8/11)	100% (0/0)
DnaRecord	100% (1/1)	57% (4/7)	72% (8/11)	100% (0/0)
exception	100% (2/2)	57% (4/7)	54% (24/44)	100% (0/0)
DnaHashCalculationException	100% (1/1)	50% (1/2)	50% (2/4)	100% (0/0)
GlobalExceptionHandler	100% (1/1)	60% (3/5)	55% (22/40)	100% (0/0)
repository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
DnaRecordRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
service	100% (3/3)	100% (16/16)	97% (103/106)	96% (85/88)
MutantDetector	100% (1/1)	100% (8/8)	100% (67/67)	96% (75/78)
MutantService	100% (1/1)	100% (4/4)	89% (25/28)	100% (6/6)
StatsService	100% (1/1)	100% (4/4)	100% (11/11)	100% (4/4)
validation	100% (1/1)	100% (4/4)	80% (21/26)	85% (12/14)
ValidDnaSequence	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
ValidDnaSequenceValidator	100% (1/1)	100% (4/4)	80% (21/26)	85% (12/14)
GlobalMutantesMercadoLibre3k9Application	0% (0/1)	0% (0/1)	0% (0/2)	100% (0/0)

Diagrama de Secuencia:

Los mismos se encuentran dentro del archivo zip con los nombres de: Diagrama de Secuencia - GET STATS.puml y Diagrama de Secuencia - POST MUTANT.puml. Puede verlo también abriendo el proyecto desde IntelliJ IDEA y buscarlo con los nombres mencionados.

PRUEBAS EN POSTMAN:

CASO MUTANTE(200)

The screenshot displays the Postman application interface. At the top, the collection is named "My Collection" and the request is labeled "Get data". The HTTP method is set to "POST" and the URL is "https://globalmutantesmercadolibre3k9.onrender.com/mutant". The "Body" tab is selected, showing a JSON payload:

```
{  "dna": ["ATGCGA", "CAGTGC", "TTATGT", "AGAAAG", "CCCCTA", "TCACTG"]}
```

. The status bar at the bottom indicates a successful "200 OK" response with a time of 3.33 s and a size of 266 B. The response body is currently empty, showing only the "Raw" tab selected.

My Collection / Get data

Save Share

POST https://globalmutantesmercadolibre3k9.onrender.com/mutant Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "dna": ["ATGCGA", "CAGTGC", "TTATGT", "AGAAAG", "CCCCTA", "TCACTG"]
3 }
```

Body Cookies Headers (9) Test Results (1/1) 200 OK • 3.33 s • 266 B Save Response

Raw Preview Visualize

1

CASO HUMANO(403)

POST

https://globalmutantesmercadolibre3k9.onrender.com/mutant

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

1

{

2

"dna":["TTGCGA","CAGTCC","TTATGT","AGAAAG","CCTCTA","TCACTG"]

3

}

Body

Cookies

Headers (9)

Test Results (0/1)

403 Forbidden

355 ms

273 B

Save Response

Raw

Preview

Debug with AI

1

STATS

GET

https://globalmutantesmercadolibre3k9.onrender.com/stats

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

This request does not have a body

Body

Cookies

Headers (12)

Test Results (1/1)

200 OK

244 ms

409 B

Save Response

JSON

Preview

Visualize

1

{

2

"count_mutant_dna": 1,

3

"count_human_dna": 1,

4

"ratio": 1.0

5

}